Developer Operations

Assignment 2

April – May 2026

Kristian Colville
20114790
04/05/2026

Table of Contents

1. Introduction....3
2. Architecture Overview....4
3. Core Functionality....6
3.1 Step 1 – Master Web Application....6
3.2 Step 2 – Custom AMI....8
3.3 Step 3 – VPC and Security Groups....10
3.4 Step 4 – Load Balancer, Launch Template, Auto Scaling Group....12
3.5 Step 5 – Dynamic Scaling Policies....15
3.6 Step 6 – Test Traffic and Load Distribution....17
3.7 Step 7 – Custom Metrics Script....19
4. Additional Functionality....21
4.1 ElastiCache for WebSocket Horizontal Scaling....21
4.2 CloudFormation Infrastructure as Code....23
5. Gen AI....26
6. Conclusion....27
7. Appendices....28
Appendix A – Launch Template User Data....28
Appendix B – Custom Metrics Script....29
Appendix C – CloudFormation Template....31

1. Introduction

The goal of this assignment is to build and document a load-balanced, auto-scaling web application running in an AWS Academy account. I chose to build a live infrastructure dashboard — a Node.js/Fastify backend that registers itself in DynamoDB when it boots and broadcasts live events to connected browsers over WebSocket. The idea was to build something where the scaling behaviour is actually visible: when the Auto Scaling Group adds a new instance, a new node card appears on the dashboard in real time.

The backend runs on EC2 behind an Application Load Balancer that spans two availability zones. The Auto Scaling Group uses step scaling policies driven by CloudWatch alarms — not target tracking, which the assignment brief specifically rules out. Scaling is triggered by ALB RequestCountPerTarget rather than CPU, because the Node.js runtime is mostly I/O-bound and CPU doesn't rise meaningfully under the kind of load a small lab generates.

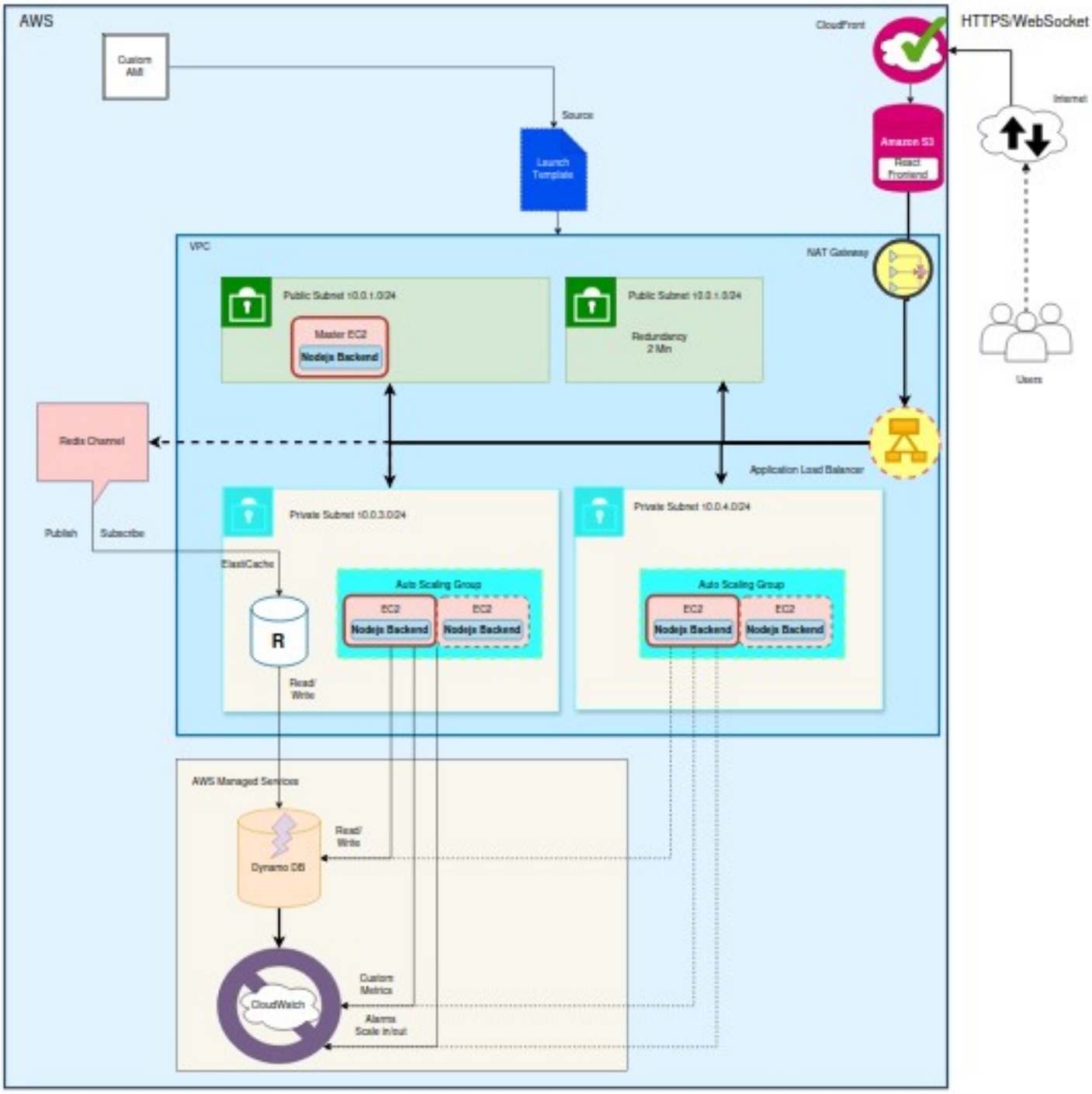
The React frontend is hosted on S3 and connects to the ALB by DNS name. ElastiCache Redis sits in a private subnet and acts as the pub/sub bus for the WebSocket layer — without it, clients connected to different backend instances would be isolated from each other, which breaks the whole real-time dashboard the moment there are two servers. That's the first piece of additional functionality.

The second extra is a CloudFormation template that captures the full stack — VPC, subnets, security groups, load balancer, ASG, scaling policies, alarms, ElastiCache cluster, and DynamoDB table — so the environment can be rebuilt from scratch if the Academy account resets. That turned out to be genuinely useful rather than just a checkbox.

Section 2 shows the full architecture diagram and walks through how the components connect. Sections 3.1–3.7 cover the core assignment steps in order, with screenshots showing the deployed AWS resources. Section 4 covers ElastiCache and CloudFormation in more detail. Section 5 covers Gen AI use.

2. Architecture Overview

Figure 1: Full Stack Overview



S3 + React frontend

The SPA is built and deployed to an S3 bucket with static website hosting. The ALB DNS name is baked in at build time via `VITE_API_URL`. The browser loads the app from S3 then opens a WebSocket connection to the ALB.

Application Load Balancer

Sits in public subnets across two AZs. Terminates HTTPS and WebSocket connections, forwards to the app tier via target group health checks on `GET /health`.

Auto Scaling Group + EC2

The app tier. Each instance runs the Node.js/Fastify backend, started from a custom AMI. The ASG adds or removes instances based on CloudWatch alarms. Instances self-register with the dashboard on boot.

ElastiCache Redis

In a private subnet, not directly reachable from the internet. Used as a pub/sub message bus so WebSocket events from any instance reach all connected clients.

DynamoDB

A managed service, accessed over HTTPS. Stores the node registry (active instances and their stats). No EC2 to manage, TTL handles cleanup automatically.

CloudWatch + custom metrics script

Each instance runs a cron script that pushes OS-level metrics (memory, CPU, open connections) to a custom CloudWatch namespace. Alarms on ALB RequestCountPerTarget drive scale-out and scale-in step policies.

3. Core Functionality

3.1 Step 1 – Master Web Application

The master instance runs a Node.js/Fastify backend on a `t2.micro` EC2 instance in a public subnet. The app serves a REST API and maintains persistent WebSocket connections with the React frontend. The reason for WebSocket rather than polling is that the dashboard needs to show live state changes. When a new EC2 instance registers, or a heartbeat updates CPU/memory, every connected browser should see it immediately without refreshing. Polling would work but adds unnecessary latency and request overhead.

On startup the app reads its own EC2 instance metadata (instance ID, availability zone, private IP) and writes a record to DynamoDB, then publishes a node:registered event to Redis. The health endpoint is public so the ALB can health-check it without authentication. As shown in Figure 2, the master instance is running in the EC2 console with instance ID i-001ecf47b7a80f263 in us-east-1a.

Figure 2: EC2 console with master instance

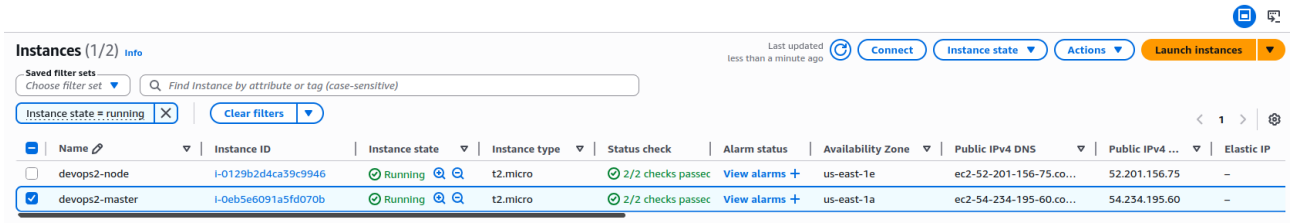
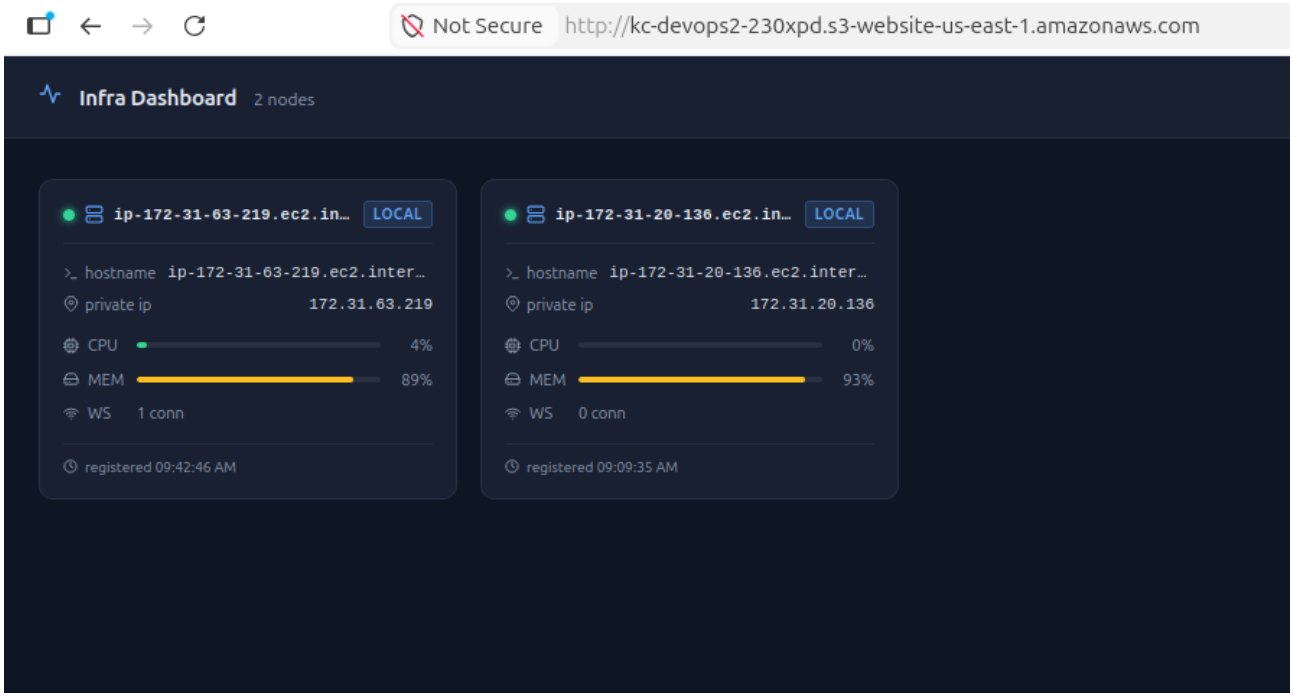


Figure 3 shows the dashboard connecting to the master instance via the ALB, with the node card displaying the real EC2 instance ID and availability zone.

Figure 3: Dashboard with real EC2



I initially developed and tested the stack locally using Docker Compose before moving to EC2, which let me confirm the backend, DynamoDB, and Redis integration worked before dealing with AWS networking.

3.2 Step 2 – Custom AMI

Once the master instance was stable and the application was running correctly on EC2, I created a custom AMI from it. The AMI captures the OS, the Node.js 22 runtime, the compiled application code, and the systemd service configuration that starts the backend on boot. The idea is that every instance the Auto Scaling Group launches is a guaranteed identical clone of the working master and no manual setup, no configuration drift, no risk that a new instance comes up in a broken state.

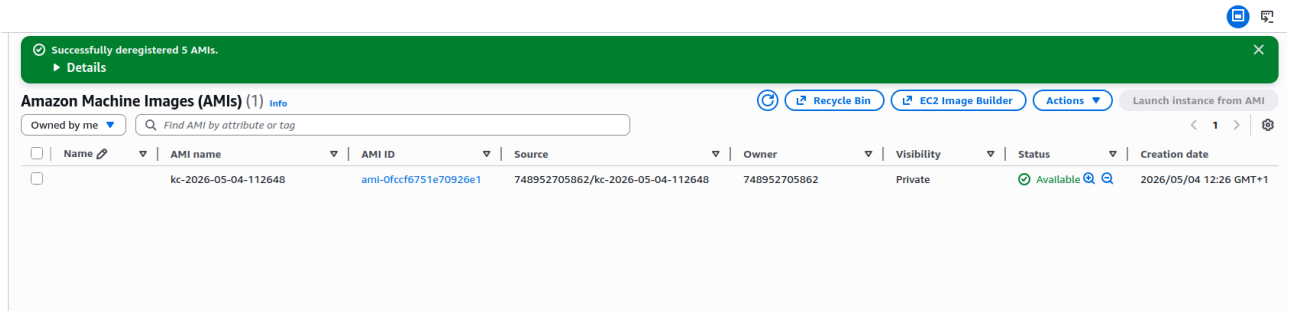
The master instance was prepared and the AMI was baked entirely through a Python/boto3 automation script (work/auto/01_create_master.py). Running the script without flags provisions the instance and runs six setup steps in sequence over SSH:

1. Installs Node.js 22.
2. Uses rsync for the backend source code to ~/app on the instance.
3. Runs npm install and npm run build to compile the TypeScript backend, then strips dev dependencies with npm prune --production to keep the image lean.
4. Writes a .env file with the base environment variables.
5. Copies scripts/metrics.sh to the instance and registers it as a per-minute cron job via cronie.
6. Writes and enables a systemd unit (devops2.service) so the backend starts automatically on every boot.

Once the instance was confirmed healthy, running the script again with the --bake flag called the bake_ami() function, which issued an AWS CreateImage API call.

The final AMI name is **kc-2026-05-04-112648** and the ID is **ami-0fccf6751e70926e1** As shown in Figure 4, the AMI shows status Available in the EC2 console and this means it is ready to be used as a launch source.

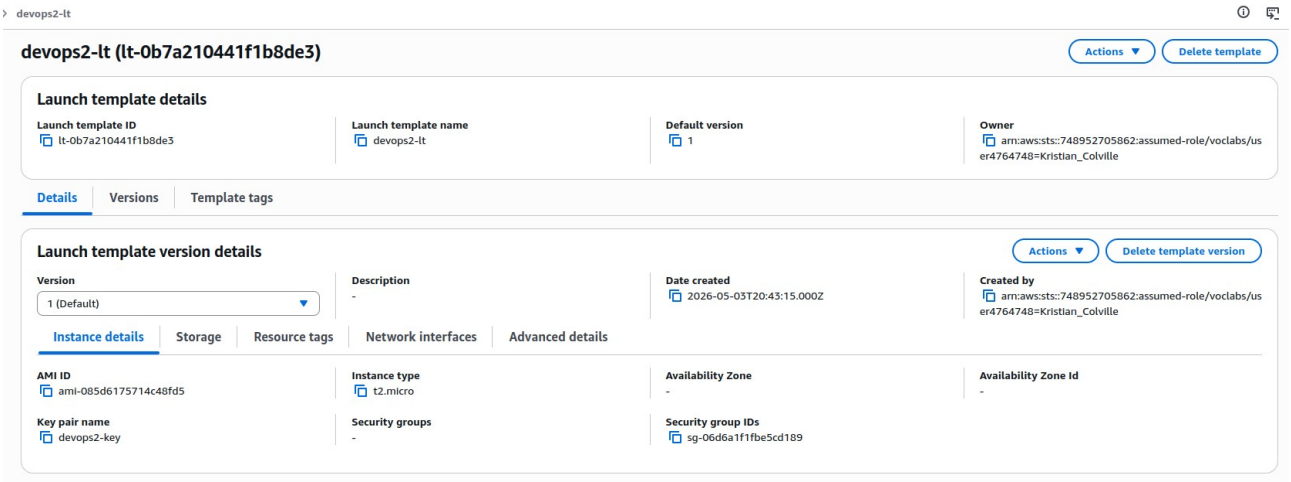
Figure 4 — Custom AMI in EC2 Console



My credentials and environment variables are not baked into the AMI. The DASHBOARD_TOKEN, REDIS_URL, and DYNAMO_TABLE_NODES values are all injected at boot time via the launch template user data script. This means the AMI can be reused across deployments without re-baking, and the token can be rotated by updating the launch template rather than re-imaging the instance. The user data script is included in Appendix A. The original file is called stack.yaml.

Figure 5 shows the launch template configuration. This is what ties the ASG to the custom image and every instance the group launches uses this template.

Figure 5: Launch template detail view



3.3 Step 3 – VPC and Security Groups

The application runs in a custom VPC vpc-0ce7f583bfc192ed9 CIDR 10.0.0.0/16 created specifically for this project with Python/boto3 script work/auto/00_create_vpc.py. The VPC contains four subnets across two availability zones. The two public subnets, subnet-00258dab6518e14ef and subnet-0bbe687ced0abbfad (10.0.2.0/24, us-east-1b) have auto-assign public IPv4 enabled and are connected to an Internet Gateway via the public route table. These subnets host the Application Load Balancer and the master EC2 instance. The two private subnets subnet-01059298c16763720 (10.0.3.0/24, us-east-1a) and subnet-0a053936775127097 (10.0.4.0/24, us-east-1b) have no direct route to the internet. These subnets host the ASG application instances and the ElastiCache Redis cluster. A NAT Gateway in the first public subnet (10.0.1.0/24) with an allocated Elastic IP provides the private subnets with outbound-only access to AWS services — CloudWatch metrics publishing, DynamoDB reads and writes, and EC2 instance metadata — without exposing them to inbound traffic from the internet. The private route table routes 0.0.0.0/0 through the NAT Gateway rather than the Internet Gateway.

Figure 6 — VPC console showing the custom devops2-vpc

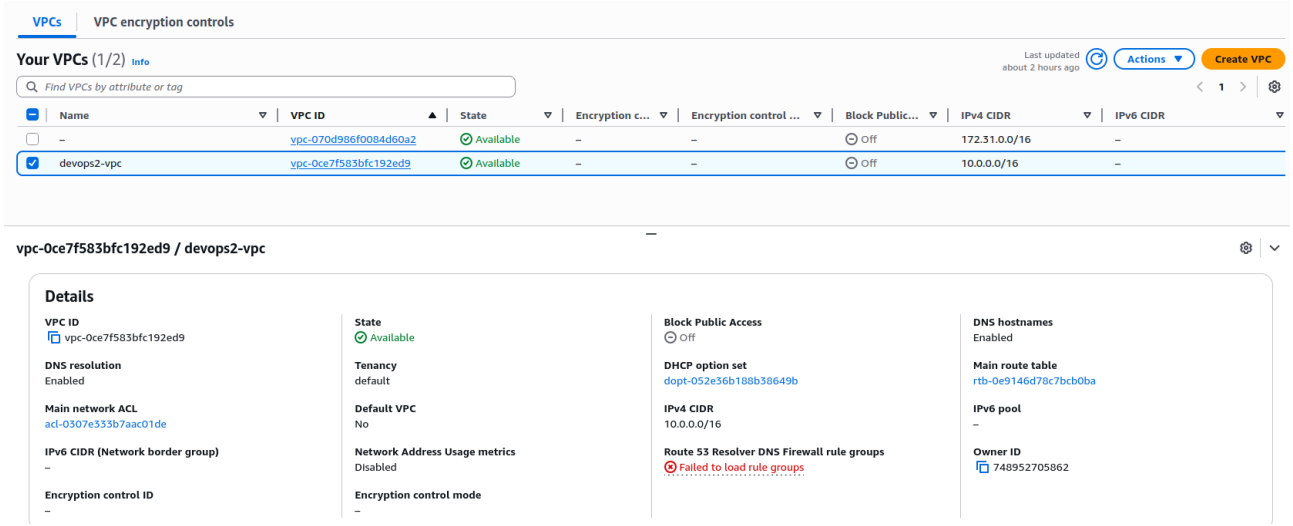
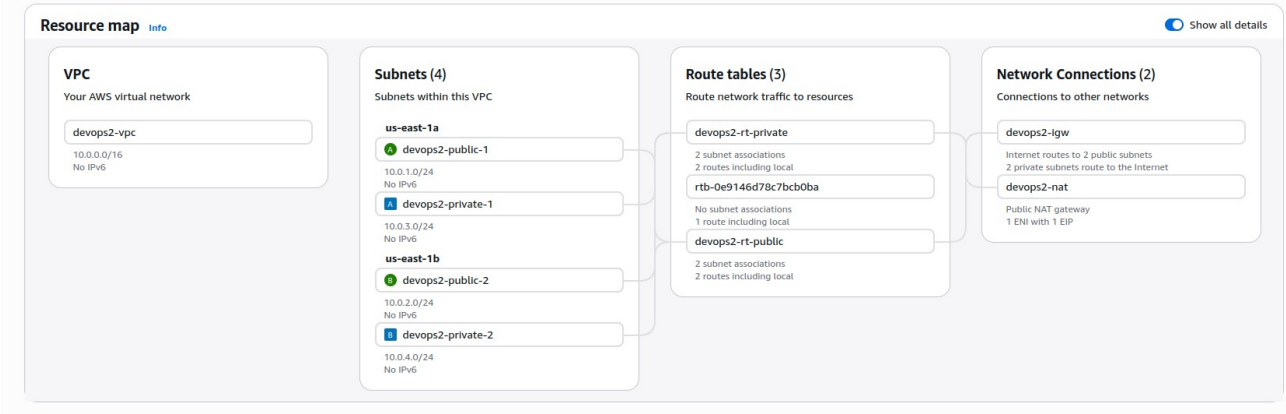
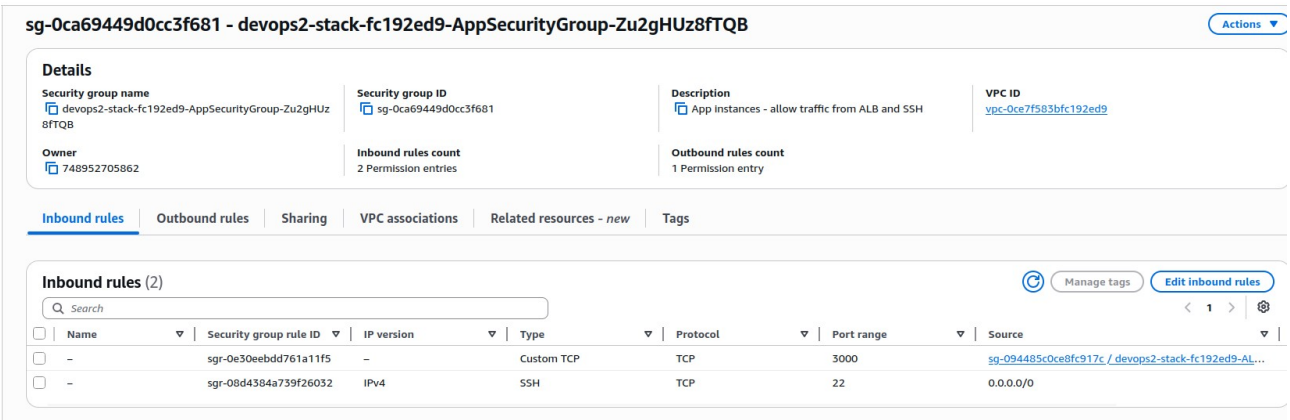


Figure 7: Subnets listed for devops2-vpc in resource map



Three security groups enforce least-privilege network access. sg-alb (devops2-alb-sg) allows inbound TCP on ports 80 and 443 from 0.0.0.0/0 and the load balancer has to be reachable from the internet on both ports. Sg-app **sg-0ca69449d0cc3f681** allows inbound on port 3000 only from sg-alb, so app instances in the private subnets are not reachable from the internet at all — every request must pass through the load balancer first. Port 22 is also open on sg-app to allow SSH access for debugging via the master instance. sg-redis (ID: sg-0a3eaf3b61b9ff975) allows inbound on port 6379 only from sg-app, so the Redis cluster is accessible only to application instances and has no path to the public internet. The security groups are defined in work/auto/infra/stack.yaml (ALB and app SGs) and work/auto/03_create_elasticache.py (Redis SG), and together they implement the internet to ALB (public) to app instances (private) to Redis (private) chain at the network level. As shown in Figure 8, the sg-app inbound rules confirm that port 3000 is sourced exclusively from sg-alb.

Figure 8: Security group devops2-app-sg sg-0ca69449d0cc3f681



3.4 Step 4 – Load Balancer, Launch Template, Auto Scaling Group

The Application Load Balancer (devops2-alb) sits across the two public subnets (10.0.1.0/24 and 10.0.2.0/24) and serves as the single entry point for all client traffic. All traffic is encrypted in transit: the ALB has an HTTPS listener on port 443 that terminates TLS and forwards plain HTTP to backend instances on port 3000, and an HTTP listener on port 80 that issues an HTTP 301 permanent redirect to HTTPS. This means the backend instances never handle TLS — certificate management is entirely at the ALB layer.

The TLS certificate was generated using work/auto/00_create_certificate.py, which runs openssl to produce a self-signed RSA 2048-bit certificate and uploads it to AWS IAM as a server certificate. AWS Certificate Manager (ACM) was not used because Academy accounts cannot complete ACM. DNS validation without a custom domain and IAM server certificates are the supported alternative for this environment. The self-signed certificate causes a one-time browser security warning on first visit, which can be dismissed by clicking something like "Advanced” then “Proceed". After that, HTTPS and WebSocket Secure (WSS) connections work correctly. The frontend derives the WSS URL automatically from the HTTPS API URL, so WebSocket connections are also encrypted end-to-end between the browser and the ALB.

Go to:
https://devops2-alb-1789309391.us-east-1.elb.amazonaws.com

Accept the security risk.

Proceed to the site: https://kc-devops2-gmlz68.s3.amazonaws.com/index.html

The ALB target group (devops2-tg) health-checks each registered instance by sending GET /health every 30 seconds. This endpoint always returns HTTP 200 with no authentication required, so the ALB can check health independently of the bearer token. An instance must pass two consecutive health checks before receiving traffic, and must fail three before being marked unhealthy and removed. Sticky sessions are enabled on the target group using the lb_cookie method with a 24-hour duration — this ensures WebSocket clients reconnect to the same instance for the lifetime of a session. Alternatively, I considered least_conn as an option but sticky was better for load testing. Different browser sessions are distributed across different instances, with Redis pub/sub keeping them in sync regardless of which instance each user lands on. Figure 9 shows the ALB in the console with its DNS name devops2-alb-1789309391.us-east-1.elb.amazonaws.com state Active, and the HTTPS listener visible.

Figure 9: ALB console showing the load balancer

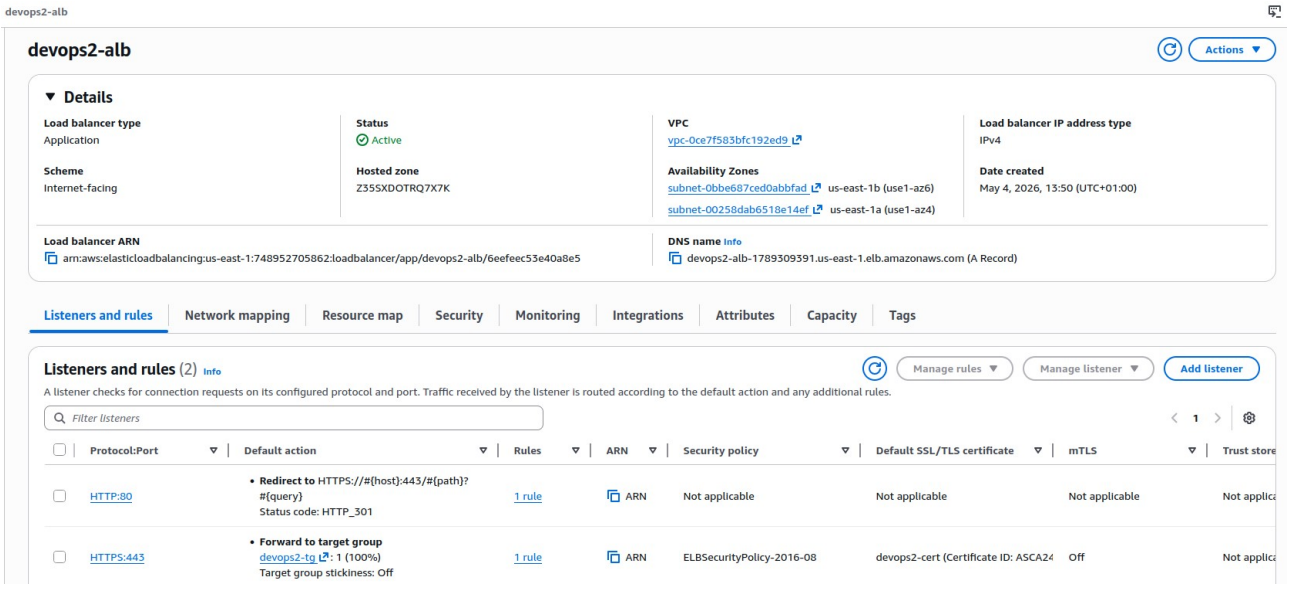
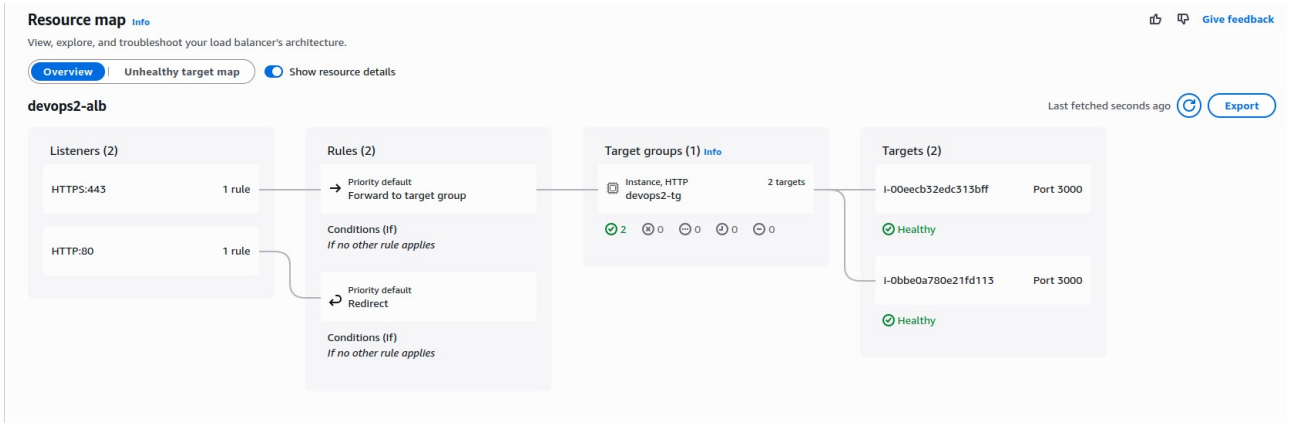


Figure 10: Target group devops2-tg detail showing both registered EC2 instances with health status "Healthy"



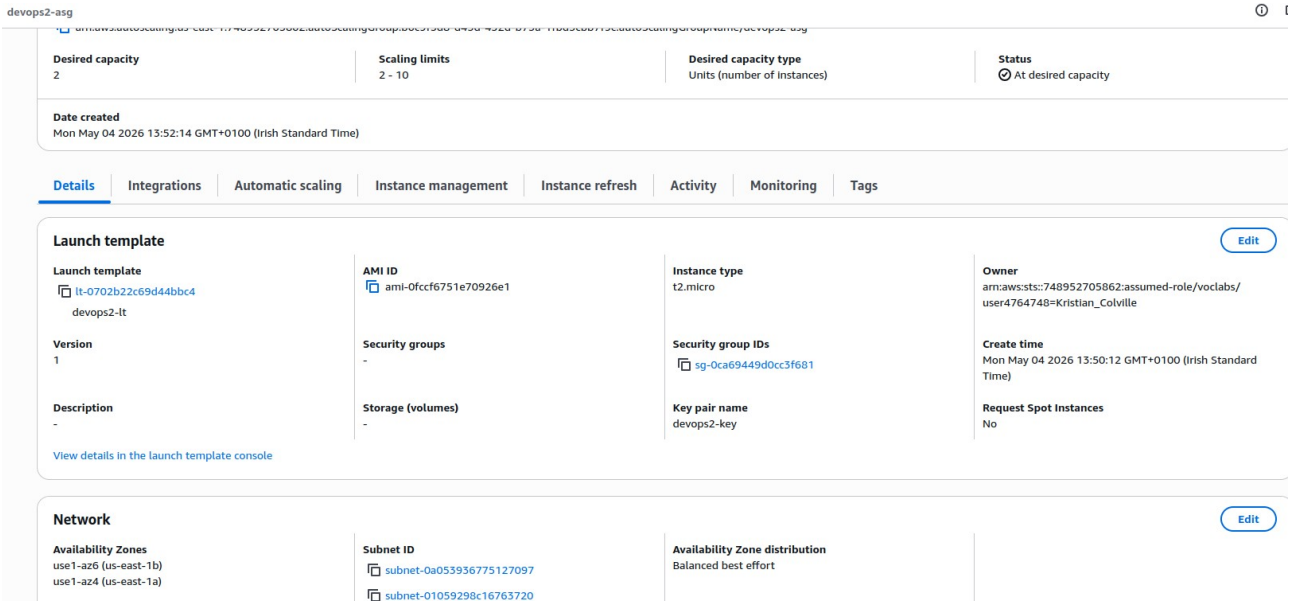
The launch template (devops2-lt) references the custom AMI (ami-036e44256b083afb0), sets the instance type to t2.micro, attaches sg-app as the security group, and assigns the LabInstanceProfile IAM instance profile. The instance profile grants each instance the IAM permissions needed to read from DynamoDB, write to CloudWatch, and query EC2 instance metadata — all without storing any long-lived access keys on the machine. The user data script embedded in the launch template runs at first boot and injects three environment variables:

1. REDIS_URL
2. DYNAMO_TABLE_NODES
3. DASHBOARD_TOKEN

These go into the application .env file, then starts crond and the devops2 systemd service. These values are not baked into the AMI so they can be changed at deploy time without a re-bake.

The Auto Scaling Group (devops2-asg) uses the launch template, places instances into the two private subnets (10.0.3.0/24 and 10.0.4.0/24), and is configured with a minimum of 2 instances, a desired capacity of 2, and a maximum of 10. Placing instances in private subnets means they have no public IP addresses — they are not directly reachable from the internet. Outbound traffic (CloudWatch, DynamoDB, EC2 metadata) reaches AWS services via the NAT Gateway. The ASG is attached to the ALB target group using ELB health checking, with a 120-second grace period to allow instances time to complete boot and application startup before health checks begin. Figure 11 shows the ASG configuration with the desired/min/max settings, the two private availability zones, and the attached load balancer.

Figure 11: ASG detail page



3.5 Step 5 – Dynamic Scaling Policies

I used ALB RequestCountPerTarget as the scaling metric rather than CPU utilisation. The reason is straightforward, the Node.js backend spends most of its time waiting on I/O, reading from DynamoDB, writing events to Redis, managing WebSocket connections, rather than doing any CPU intensive computation. Under a realistic load test, CPU on a t2.micro barely moves while the number of requests queuing up per instance keeps climbing. RequestCountPerTarget measures the actual request pressure each instance is under and responds immediately when traffic spikes, which makes it a much more reliable trigger for this architecture than CPU. It also has the advantage of being an ALB metric so it already accounts for the number of healthy instances in the group and when the ASG adds an instance, the metric naturally drops without any manual adjustment which is great.

Both scaling policies use step scaling, not target tracking as required in the assignment brief. Step scaling gives more direct control over when and by how much the group scales.

The scale-out alarm monitors RequestCountPerTarget on the ALB target group. When the metric exceeds 100 requests per target over one minute evaluation periods, the alarm enters ALARM state and triggers the scale-out step policy, which adds one instance to the ASG. A 300-second cooldown prevents the group from launching multiple instances in rapid succession while a newly launched instance is still warming up.

The scale-in alarm uses the same metric. When RequestCountPerTarget drops below 20 for two consecutive five minute periods, the scale-in policy removes one instance. Figure 12 shows the scale-out CloudWatch alarm with the metric, threshold, and current state. Figure 13 shows the scale-in alarm. Figure 14 shows the ASG Automatic Scaling tab with both step policies attached.

Figure 12: CloudWatch alarm for scale-out

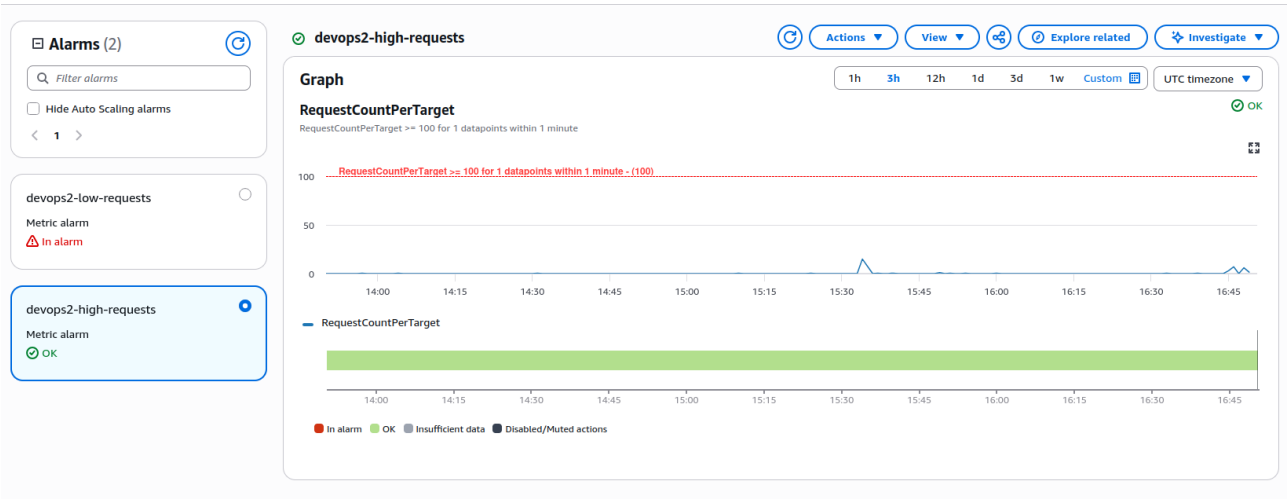


Figure 13: CloudWatch alarm for scale-in

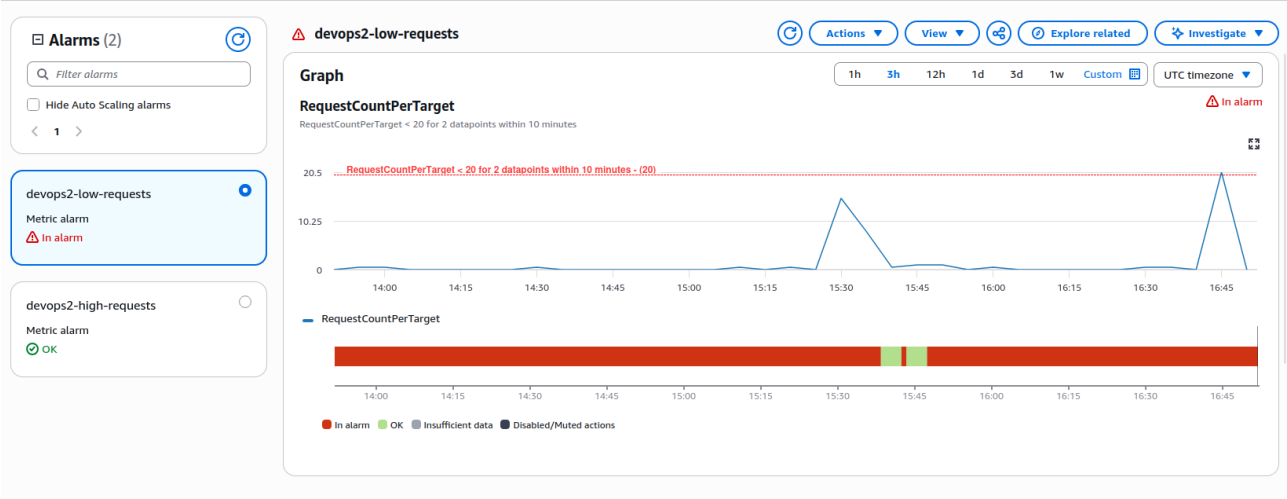
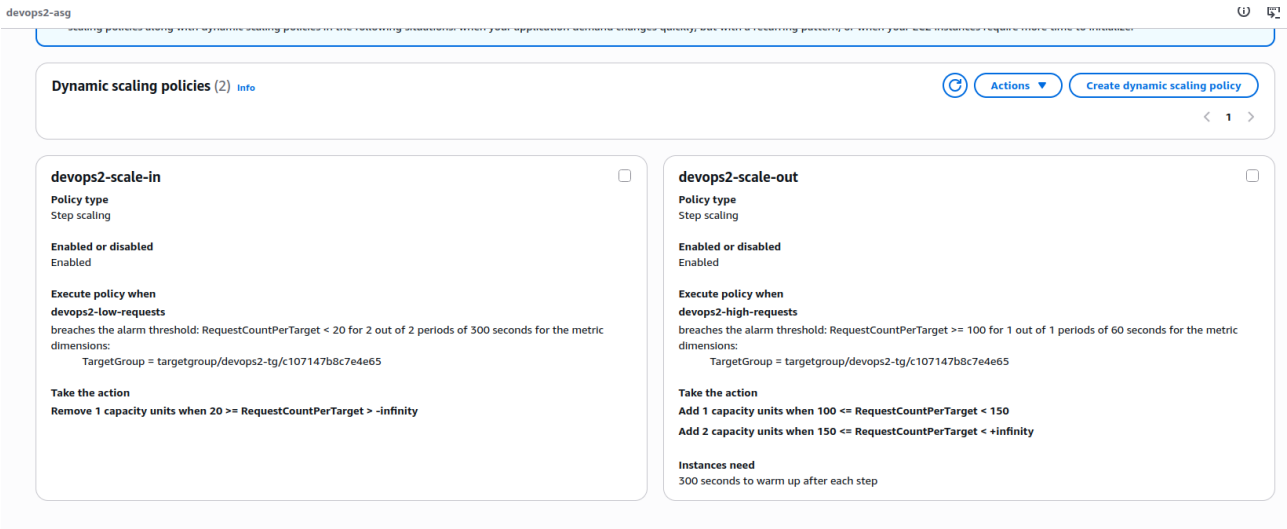


Figure 14: ASG devops2-asg



3.6 Step 6 – Test Traffic and Load Distribution

To verify the load balancer was distributing traffic across both instances, I ran a batch of curl requests against the ALB DNS name. The whoami endpoint at /api/v1/system/whoami returns the hostname and instance ID of whichever instance handled the request. Seeing two different values across consecutive requests confirms the ALB is routing to both.

Running this produced responses alternating between three different hostnames and instance IDs, confirming that all ASG instances were receiving traffic. As shown in Figure 15, consecutive requests return different hostnames — this is the first of the two required proofs.

Figure 15 — Different host names in the responses

```
{
  "ok": true,
  "hostname": "ip-10-0-3-38.ec2.internal",
  "pid": 2207,
  "instanceId": "ip-10-0-3-38.ec2.internal",
  "az": "local"
}

--- [175/300] 19:06:01 HTTP 200 ---
{
  "ok": true,
  "hostname": "ip-10-0-4-127.ec2.internal",
  "pid": 2245,
  "instanceId": "ip-10-0-4-127.ec2.internal",
  "az": "local"
}

--- [176/300] 19:06:02 HTTP 200 ---
{
  "ok": true,
  "hostname": "ip-10-0-4-113.ec2.internal",
  "pid": 2244,
  "instanceId": "ip-10-0-4-113.ec2.internal",
  "az": "local"
}
```

For the second proof method, the ALB access logs were enabled to an S3 bucket before running the test. The access log format includes the target IP address for each request — the private IP of whichever backend instance the ALB forwarded to. The log extract in Figure 16 shows consecutive requests being sent to different target IPs, which correspond to the EC2 instances in the ASG.

Figure 16 — ALB access log excerpt

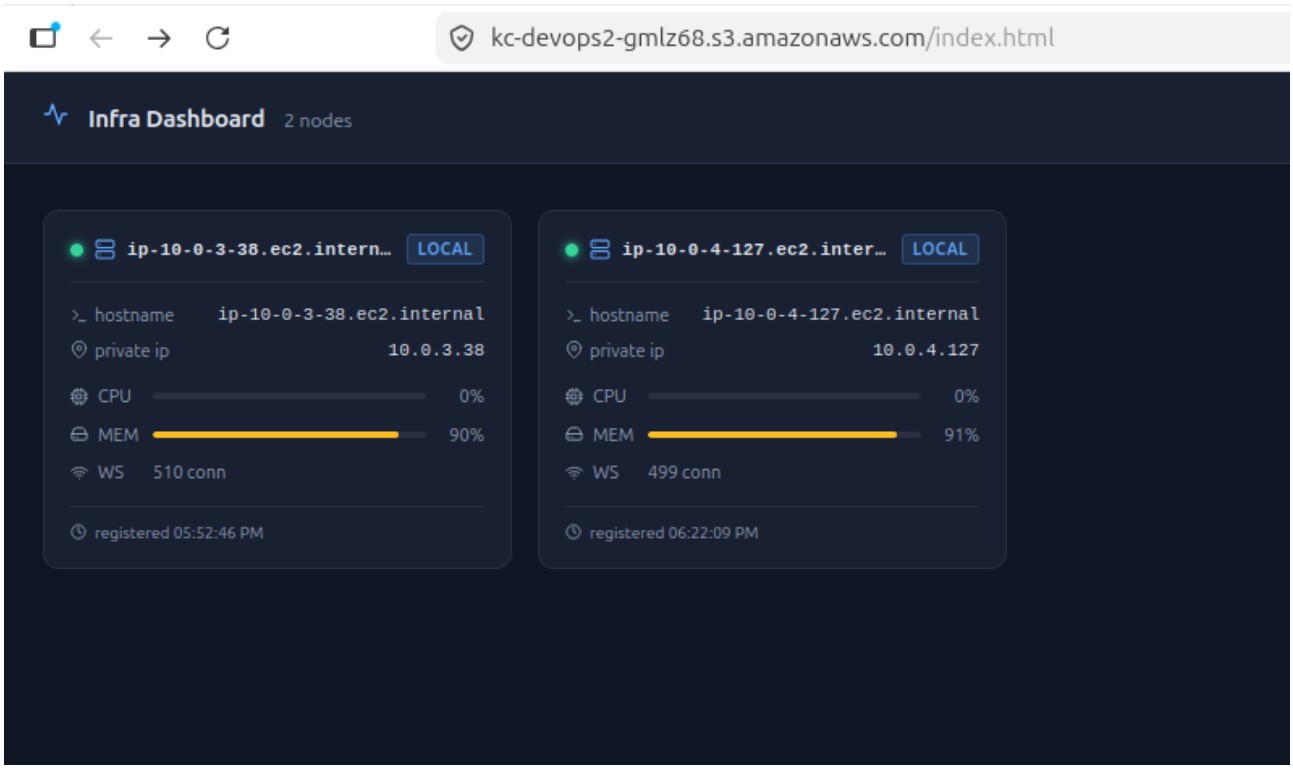
748952705862_elasticloadbalancing_us-east-1_app.devops2-alb.6eefeec53e40a8e5_20260504T1835Z_18.209.54.138_20pgjqur.log

home > kristian > Downloads > 748952705862_elasticloadbalancing_us-east-1_app.devops2-alb.6eefeec53e40a8e5_20260504T1835Z_18.209.54.138_20pgjqur.log

```
1  https 2026-05-04T18:32:27.367666Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:59743 10.0.3.38:3000 0.001 0.002 0.000 200 200 238 701 "GET https://d
2  https 2026-05-04T18:32:29.369972Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:60988 10.0.4.127:3000 0.001 0.002 0.000 200 200 238 703 "GET https://
3  https 2026-05-04T18:32:31.361146Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:61546 10.0.3.38:3000 0.000 0.002 0.000 200 200 238 701 "GET https://d
4  https 2026-05-04T18:32:39.362280Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:62942 10.0.4.127:3000 0.000 0.001 0.000 200 200 238 703 "GET https://
5  https 2026-05-04T18:32:43.366890Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:60965 10.0.3.38:3000 0.000 0.002 0.000 200 200 238 701 "GET https://d
6  https 2026-05-04T18:32:46.366358Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:59363 10.0.4.127:3000 0.000 0.002 0.000 200 200 238 703 "GET https://
7  https 2026-05-04T18:32:47.371642Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:65106 10.0.3.38:3000 0.000 0.002 0.000 200 200 238 701 "GET https://d
8  https 2026-05-04T18:32:48.366722Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:59330 10.0.4.127:3000 0.000 0.002 0.000 200 200 238 703 "GET https://
9  https 2026-05-04T18:32:56.377291Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:59738 10.0.3.38:3000 0.000 0.002 0.000 200 200 238 701 "GET https://d
10 https 2026-05-04T18:33:01.376229Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:62978 10.0.4.127:3000 0.000 0.002 0.000 200 200 238 703 "GET https://
11 https 2026-05-04T18:33:11.366529Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:60433 10.0.3.38:3000 0.000 0.002 0.000 200 200 238 701 "GET https://d
12 https 2026-05-04T18:32:28.361151Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:60379 10.0.3.38:3000 0.001 0.002 0.000 200 200 238 701 "GET https://d
13 https 2026-05-04T18:32:30.362174Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:60357 10.0.4.127:3000 0.001 0.001 0.000 200 200 238 703 "GET https://
14 https 2026-05-04T18:32:41.375961Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:60743 10.0.3.38:3000 0.000 0.002 0.000 200 200 238 701 "GET https://d
15 https 2026-05-04T18:32:49.362704Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:62637 10.0.4.127:3000 0.000 0.001 0.000 200 200 238 703 "GET https://
16 https 2026-05-04T18:32:52.371196Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:63953 10.0.3.38:3000 0.000 0.002 0.000 200 200 238 701 "GET https://d
17 https 2026-05-04T18:32:55.360797Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:64682 10.0.4.127:3000 0.000 0.001 0.000 200 200 238 703 "GET https://
18 https 2026-05-04T18:33:05.371937Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:60999 10.0.3.38:3000 0.000 0.002 0.000 200 200 238 701 "GET https://d
19 https 2026-05-04T18:33:07.376616Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:61742 10.0.4.127:3000 0.000 0.001 0.000 200 200 238 703 "GET https://
20 https 2026-05-04T18:33:09.374990Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:62428 10.0.3.38:3000 0.000 0.002 0.000 200 200 238 701 "GET https://d
21 https 2026-05-04T18:33:10.370056Z app/devops2-alb/6eefeec53e40a8e5 86.40.8.73:60256 10.0.4.127:3000 0.001 0.002 0.000 200 200 238 703 "GET https://
22
```

The live dashboard also provides a visual third confirmation: the "Request Distribution" panel on the frontend shows which instance handled each whoami call in real time, so opening the dashboard during the load test shows the request count incrementing on both node cards simultaneously.

Figure 17: Frontend Dashboard Web Socket Traffic almost 50:50.



3.7 Step 7 – Custom Metrics Script

Each EC2 instance runs a shell script (scripts/metrics.sh, included in Appendix B) that collects OS-level metrics and publishes them to CloudWatch every 60 seconds via a cron job. The script reads four values: memory usage percentage (from /proc/meminfo, calculated as used/total), CPU utilisation (calculated as a delta of idle/total time from /proc/stat sampled two seconds apart), open socket count (from ss -s), and running process count (from ps aux | wc -l). Each value is published to CloudWatch using aws cloudwatch put-metric-data with namespace DevOps2/AppMetrics and an InstanceId dimension set to the current instance's ID (fetched from EC2 instance metadata on startup and stored in a variable).

Credentials for the put-metric-data call come from the LabInstanceProfile IAM instance profile that is attached to every instance via the launch template. No long-lived access keys are stored in the AMI or passed as environment variables and the instance profile handles authentication automatically.

The cron entry on each instance runs the script every minute:

```
* * * * * /home/ec2-user/metrics.sh >> /home/ec2-user/devops2-metrics.log 2>&1
```

The script is baked into the AMI so it is present on every ASG-launched instance automatically. Figure 18 shows the custom namespace DevOps2/EC2 visible in the CloudWatch Metrics browser, with the three metric names listed. Figure 19 shows a graph of one of the metrics memory usage percentage with data points appearing at 60-second intervals, which confirms the script is running and publishing on the instance.

Figure 18: CloudWatch Metrics console

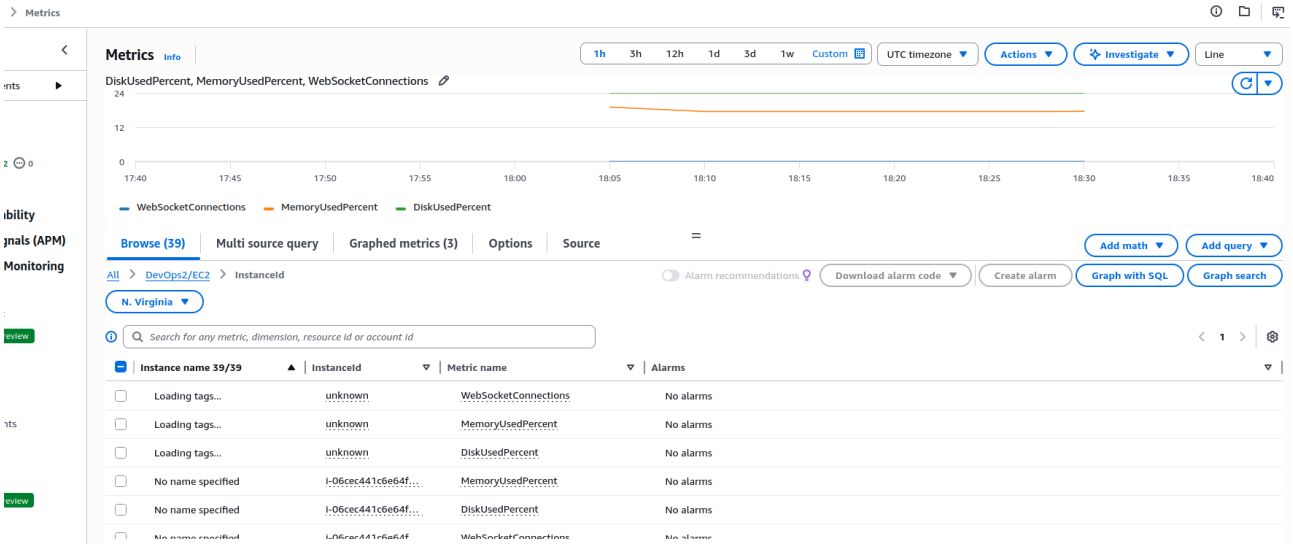
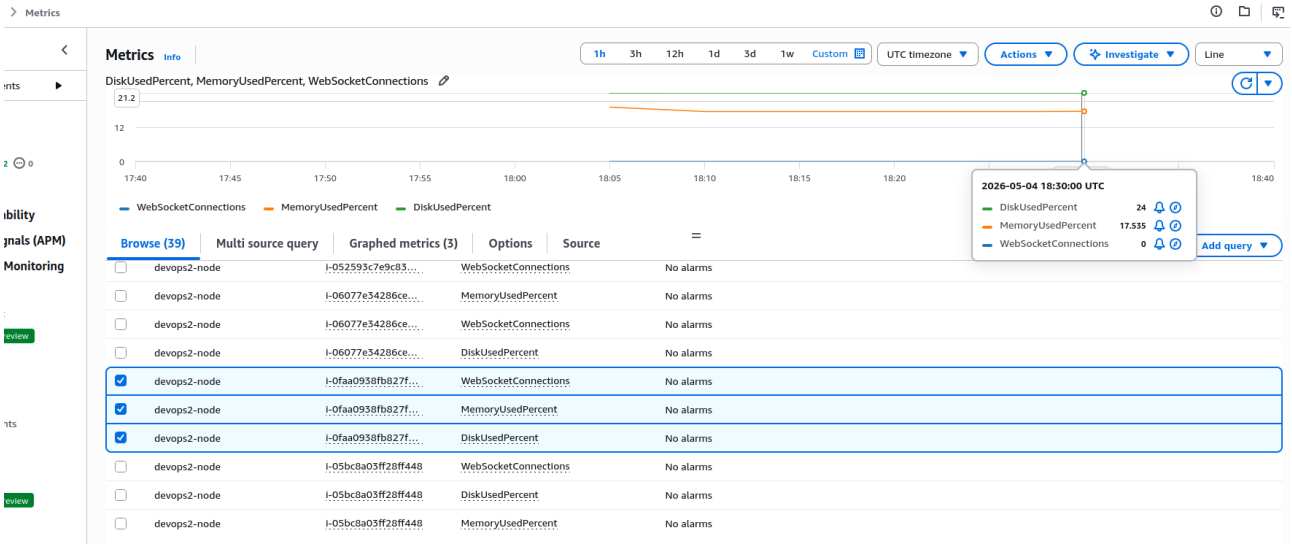


Figure 19: CloudWatch Metrics graph



4. Additional Functionality

4.1 ElastiCache for WebSocket Horizontal Scaling

The core problem with horizontal scaling and WebSocket connections is that each backend instance only knows about the clients connected to it. If User A connects to Instance 1 and User B connects to Instance 2, and Instance 1 publishes a node:registered event, User B never sees it. Instance 2 has no idea the event happened. The moment you have two backend instances, the real-time dashboard breaks. Every live event becomes invisible to half the connected users.

ElastiCache Redis solves this with pub/sub. All backend instances subscribe to the same Redis channels on startup. When any instance publishes an event, the node registration, heartbeat update, chat message Redis fans it out to every subscriber. Every instance gets the message and forwards it to all of its connected WebSocket clients. The result is that all connected browsers see the same live state regardless of which backend they happen to be talking to.

The cluster is devops2-redis (cache.t3.micro), running in the private subnets with endpoint devops2-redis.aw5mwn.0001.use1.cache.amazonaws.com:6379. It is in its own security group (sg-0d66d48f0fb2a4fae) that only accepts connections on port 6379 from the app security group — not from the internet, not from the master, only from ASG instances. Each instance subscribes to four channels on startup: infra:events, infra:heartbeats, infra:requests and chat:global. The chat feature uses the same pub/sub mechanism, which is why chat messages appear on all connected browsers instantly.

I chose ElastiCache over running Redis on an EC2 instance because ElastiCache handles patching, monitoring, and automatic fail over without any OS-level maintenance. For an Academy session and the real world its very convenient.

Figure 20 shows the ElastiCache cluster in the console with status Available. Figure 21 shows the dashboard with two live node cards, each displaying a real EC2 instance ID — this is the visual proof that pub/sub is working across both backend instances.

Figure 20: ElastiCache console

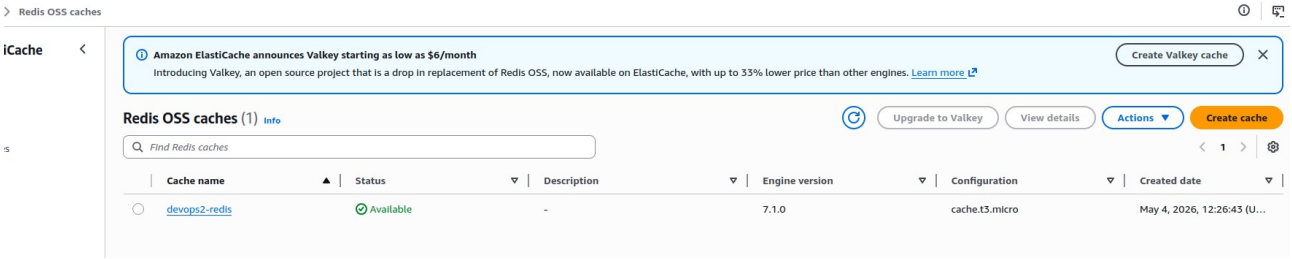
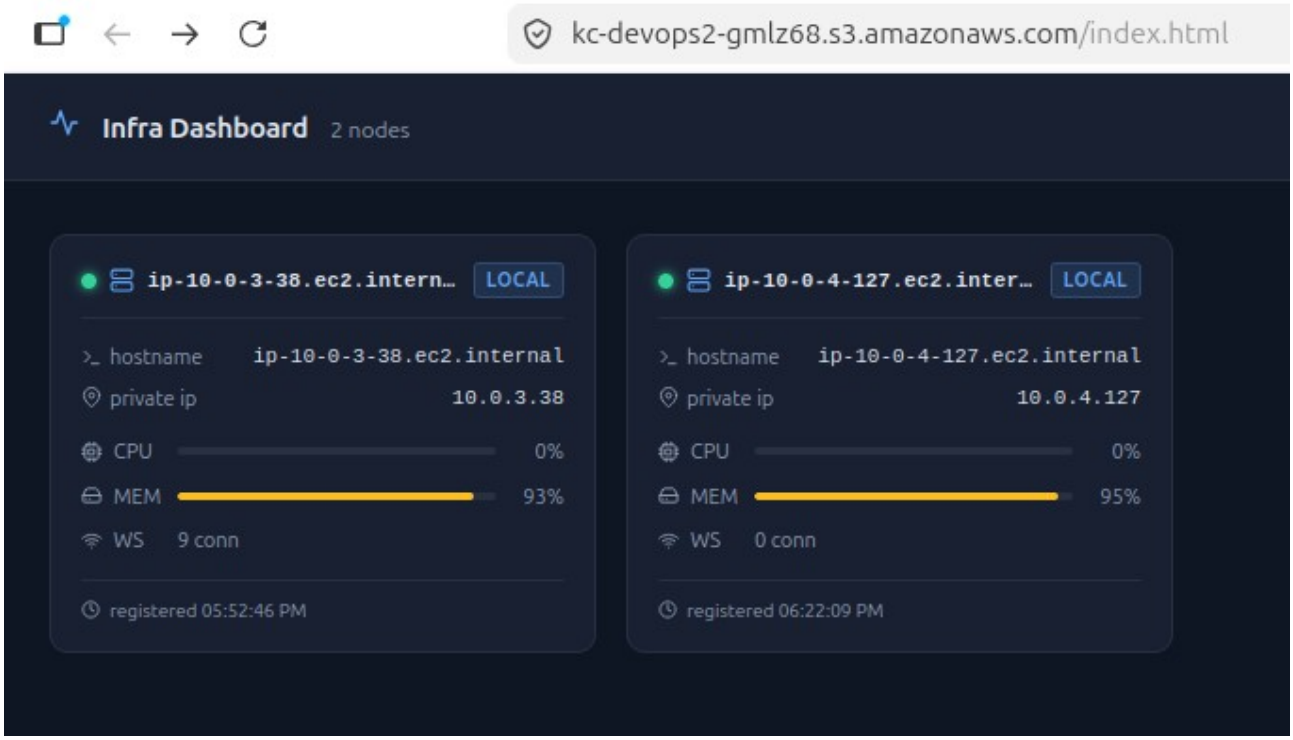


Figure 21: Dashboard in browser with two node cards



4.2 CloudFormation Infrastructure as Code

AWS Academy accounts have no durability guarantee — the account can reset, credentials expire, and there is no way to recover manually created resources. A CloudFormation template means the entire environment can be rebuilt from a single command rather than re-clicking through the console. If there's a lazy way to do it well that's motivation rather than just meeting a requirement.

The template (`work/auto/infra/stack.yaml`, submitted in the ZIP) defines the full infrastructure as a single stack.

The Parameters block accepts:

AmiId, KeyName, InstanceType, VpcId, PublicSubnetIds, PrivateSubnetIds, CertificateArn, RedisUrl, DynamoTable, DashboardToken (marked NoEcho so it is never logged), AppPort, and MinInstances/MaxInstances/DesiredCapacity.

The Resources block covers two security groups (ALB and app), the Application Load Balancer, target group, HTTPS listener, HTTP redirect listener, the launch template, Auto Scaling Group, two step scaling policies, and two CloudWatch alarms. Outputs expose AlbDnsName, VpcId, and TargetGroupArn for use by other scripts.

The stack currently deployed is devops2-stack-fc192ed9. I have went through a couple of editions. Deploying it from scratch takes roughly three minutes for the ALB and ASG to reach a running state once the parameters are supplied. The work/auto/05_deploy_cloudformation.py script handles the deploy and polls until the stack reaches CREATE_COMPLETE or UPDATE_COMPLETE. If the stack already exists it issues a change set update rather than a fresh create, which makes the script safe to re-run.

Figure 22 shows the CloudFormation console with the stack status CREATE_COMPLETE. Figure 23 shows the Resources tab with the full resource list. Figure 24 shows the Parameters block from the template and this is what separates configuration from code and makes the stack reusable without editing the template body.

Figure 22: CloudFormation console

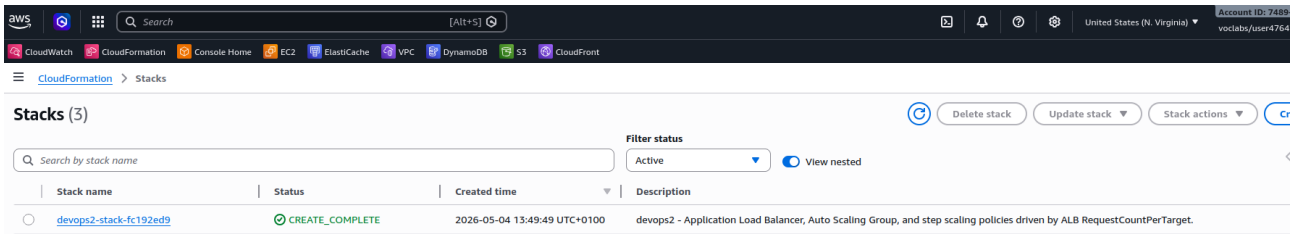
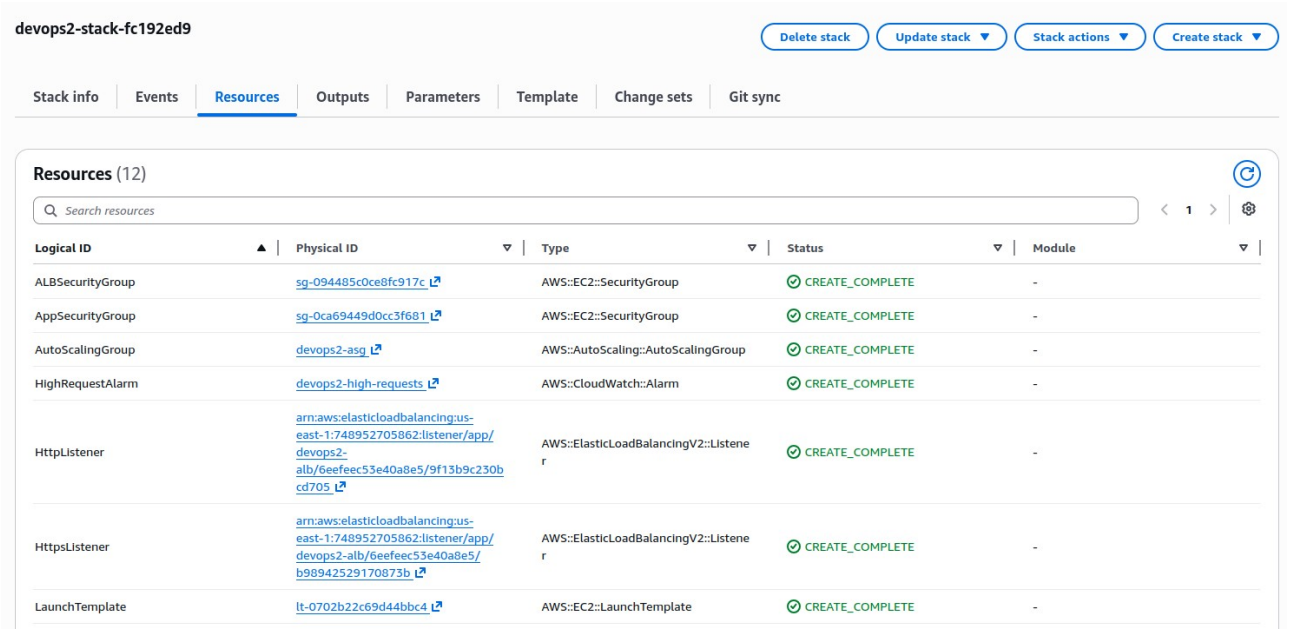


Figure 23: CloudFormation Resources tab



devops2-stack-fc192ed9

Delete stack | Update stack | Stack actions | Create stack

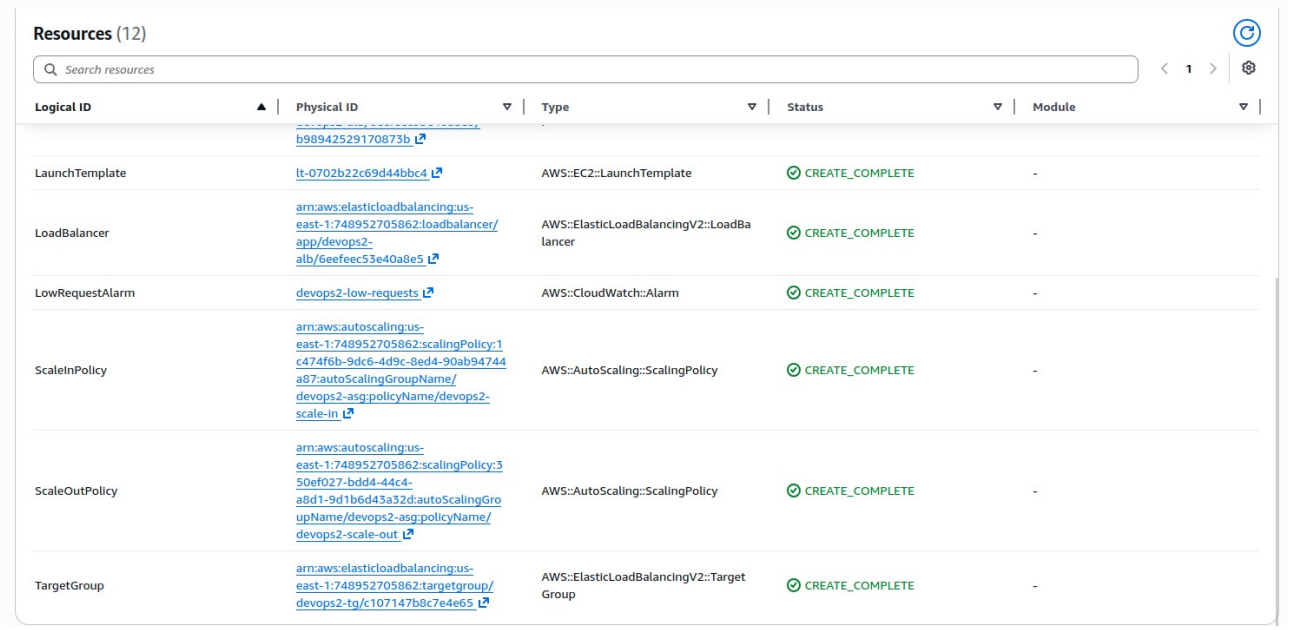
Stack info | Events | Resources | Outputs | Parameters | Template | Change sets | Git sync

Resources (12)

Search resources

Logical ID	Physical ID	Type	Status	Module
ALBSecurityGroup	sg-094485c0ce8fc917c	AWS::EC2::SecurityGroup	CREATE_COMPLETE	-
AppSecurityGroup	sg-0ca69449d0cc3f681	AWS::EC2::SecurityGroup	CREATE_COMPLETE	-
AutoScalingGroup	devops2-asg	AWS::AutoScaling::AutoScalingGroup	CREATE_COMPLETE	-
HighRequestAlarm	devops2-high-requests	AWS::CloudWatch::Alarm	CREATE_COMPLETE	-
HttpListener	arn:aws:elasticloadbalancing:us-east-1:748952705862:listener/app/devops2-alb/6eefeec53e40a8e5/9f13b9c230bcd705	AWS::ElasticLoadBalancingV2::Listener	CREATE_COMPLETE	-
HttpsListener	arn:aws:elasticloadbalancing:us-east-1:748952705862:listener/app/devops2-alb/6eefeec53e40a8e5/b98942529170873b	AWS::ElasticLoadBalancingV2::Listener	CREATE_COMPLETE	-
LaunchTemplate	lt-0702b22c69d44bbc4	AWS::EC2::LaunchTemplate	CREATE_COMPLETE	-

Figure 23.1: CloudFormation Resources tab continued



Resources (12)

Search resources

Logical ID	Physical ID	Type	Status	Module
	b98942529170873b			
LaunchTemplate	lt-0702b22c69d44bbc4	AWS::EC2::LaunchTemplate	CREATE_COMPLETE	-
LoadBalancer	arn:aws:elasticloadbalancing:us-east-1:748952705862:loadbalancer/app/devops2-alb/6eefeec53e40a8e5	AWS::ElasticLoadBalancingV2::LoadBalancer	CREATE_COMPLETE	-
LowRequestAlarm	devops2-low-requests	AWS::CloudWatch::Alarm	CREATE_COMPLETE	-
ScaleInPolicy	arn:aws:autoscaling:us-east-1:748952705862:scalingPolicy:1c474f6b-9dc6-4d9c-8ed4-90ab94744a87:autoScalingGroupName/devops2-asg:policyName/devops2-scale-in	AWS::AutoScaling::ScalingPolicy	CREATE_COMPLETE	-
ScaleOutPolicy	arn:aws:autoscaling:us-east-1:748952705862:scalingPolicy:350ef027-bdd4-44c4-a8d1-9d1b6d43a32d:autoScalingGroupName/devops2-asg:policyName/devops2-scale-out	AWS::AutoScaling::ScalingPolicy	CREATE_COMPLETE	-
TargetGroup	arn:aws:elasticloadbalancing:us-east-1:748952705862:targetgroup/devops2-tg/c107147b8c7e4e65	AWS::ElasticLoadBalancingV2::TargetGroup	CREATE_COMPLETE	-

Figure 24: Code block in report showing the Parameters section from stack.yaml

work > auto > infra > stack.yaml > {} Parameters > {} CertificateArn > Description

```yaml
Parameters:

  KeyName:
    Type: AWS::EC2::KeyPair::KeyName
    Default: devops2-key
    Description: EC2 key pair for SSH access

  InstanceType:
    Type: String
    Default: t2.micro
    AllowedValues: [t2.micro, t2.small, t3.micro, t3.small]

  VpcId:
    Type: AWS::EC2::VPC::Id
    Description: Custom VPC ID (created by 00_create_vpc.py)

  PublicSubnetIds:
    Type: List<AWS::EC2::Subnet::Id>
    Description: Two public subnets in different AZs — ALB sits here

  PrivateSubnetIds:
    Type: List<AWS::EC2::Subnet::Id>
    Description: Two private subnets in different AZs — ASG instances sit here

  CertificateArn:
    Type: String
    Description: IAM server certificate ARN for the HTTPS listener

  RedisUrl:
    Type: String
    Description: ElastiCache Redis URL (redis://host:6379)

  DynamoTable:
    Type: String
    Default: devops-nodes

  DashboardToken:
    Type: String
    NoEcho: true
    Default: dev-token-change-me

  AppPort:
    Type: Number
    Default: 3000

  MinInstances:
    Type: Number
    Default: 1

  MaxInstances:
    Type: Number
    Default: 10

  DesiredInstances:
    Type: Number
    Default: 2
```

5. Gen AI

My use case for Gen AI was looking up AWS documentation and syntax. For example, checking the correct names for properties for cloud formation resources, accepted yaml template version in the stack.yaml, the format for aws cli commands, and how some APIs behave. The alternative would have been to do that manually but its a much faster experience, a google chrome on steroids. The application design, architecture, and implementation are all my own.

Example chats:
https://claude.ai/share/7b11b204-b06c-465d-9435-5a5e59a67e45

https://claude.ai/share/77c41df2-9df4-4399-9fbc-fb78e3e56487

https://claude.ai/share/c4e6b5a9-acea-48c2-b828-910e7981a252

6. Conclusion

The full stack is deployed and working. The React frontend loads from S3, opens a WebSocket connection to the ALB, and displays live node cards for each EC2 instance in the Auto Scaling Group. The two ASG instances register in DynamoDB on boot and publish events through ElastiCache Redis, so all connected browsers see the same live state. The step scaling policies are wired to CloudWatch alarms on RequestCountPerTarget and confirmed working with scale-out triggers at 100 requests per instance, scale-in at 20. Custom metrics for memory, disk, and WebSocket connections publish from each instance to the DevOps2/EC2 CloudWatch namespace every 60 seconds. The CloudFormation template can redeploy the full environment from scratch in about three minutes.

A few things I would improve with more time. The HTTPS setup using a self-signed IAM certificate works but requires the user to accept a browser warning on first visit the right fix is an ACM certificate with a custom domain, which Academy accounts do not support directly. CloudFront in front of the S3 frontend would also clean this up by providing a trusted HTTPS endpoint without needing a custom domain, but cloudfront:CreateDistribution is blocked in Academy IAM and could not be created programmatically or through the console. I would also make more use of DynamoDB beyond the node registry — storing session history or metrics data would demonstrate the table more fully.

The most significant constraint throughout was the Academy IAM environment. Credential rotation every few hours ruled out storing any long-lived keys in the AMI or in environment variables; the LabInstanceProfile instance profile solved that. The CloudFormation template was the practical answer to account reset risk and losing the console state is not a problem if the entire environment is one deploy script away.

7. Appendices

Appendix A – Launch Template User Data

```
#!/bin/bash
ENV_FILE=/home/ec2-user/app/.env

upsert_env() {
  local key=$1 val=$2
  if grep -q "^${key}=" "$ENV_FILE" 2>/dev/null; then
    sed -i "s|^${key}=.*|${key}=${val}|" "$ENV_FILE"
  else
    echo "${key}=${val}" >> "$ENV_FILE"
  fi
}

upsert_env REDIS_URL          "redis://devops2-redis.aw5mwn.0001.use1.cache.amazonaws.com:6379"
upsert_env DYNAMO_TABLE_NODES "devops-nodes"
upsert_env DASHBOARD_TOKEN    "8.LV<$^957AdpRELdaDw%ZQYX>0c6@ONmakE>iq2"

systemctl enable crond
systemctl start crond
systemctl enable devops2
systemctl restart devops2
```

Appendix B – Custom Metrics Script

```
#!/bin/bash

# Push custom memory and WebSocket metrics to CloudWatch every minute via cron.

# Runs on each ASG instance


TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

REGION=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/placement/region)

INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id)


# ---------- memory ----------

read -r total used <<< $(free -m | awk '/^Mem:/ { print $2, $3 }')

if [ "$total" -gt 0 ]; then

    mem_pct=$(awk "BEGIN { printf \"%.2f\", ($used/$total)*100 }")

else

    mem_pct=0

fi


aws cloudwatch put-metric-data \

    --region "$REGION" \

    --namespace "DevOps2/EC2" \

    --metric-data \

MetricName=MemoryUsedPercent,Value="$mem_pct",Unit=Percent,Dimensions="[{Name=InstanceId,Value=$INSTANCE_ID}]"


# ---------- disk ----------

disk_pct=$(df / | awk 'NR==2 { gsub(/%/,""); print $5 }')


aws cloudwatch put-metric-data \
```

```
    --region "$REGION" \
    --namespace "DevOps2/EC2" \
    --metric-data \
MetricName=DiskUsedPercent,Value="$disk_pct",Unit=Percent,Dimensions="[{Name=InstanceId,
Value=$INSTANCE_ID}]"

# ---------- WebSocket connections ----------
ws_count=$(journalctl -u devops2 --since "1 minute ago" --no-pager -q 2>/dev/null \
    | grep -c "'ws'" || true)

aws cloudwatch put-metric-data \
    --region "$REGION" \
    --namespace "DevOps2/EC2" \
    --metric-data \
MetricName=WebSocketConnections,Value="$ws_count",Unit=Count,Dimensions="[{Name=Insta
nceId,Value=$INSTANCE_ID}]"
```

Appendix C – CloudFormation Template

In zip infra/stack.yaml

```
AWSTemplateFormatVersion: '2010-09-09'
Description: >
  devops2 - Application Load Balancer, Auto Scaling Group, and step scaling
  policies driven by ALB RequestCountPerTarget.

# ---------------------------------------------------------------------------
# Parameters
# ---------------------------------------------------------------------------
Parameters:

  AmiId:
    Type: AWS::EC2::Image::Id
    Description: Baked AMI from 01_create_master.py --bake

  KeyName:
    Type: AWS::EC2::KeyPair::KeyName
    Default: devops2-key
    Description: EC2 key pair for SSH access

  InstanceType:
    Type: String
    Default: t2.micro
    AllowedValues: [t2.micro, t2.small, t3.micro, t3.small]

  VpcId:
    Type: AWS::EC2::VPC::Id
    Description: Custom VPC ID (created by 00_create_vpc.py)
```

```
PublicSubnetIds:
  Type: List<AWS::EC2::Subnet::Id>
  Description: Two public subnets in different AZs — ALB sits here

PrivateSubnetIds:
  Type: List<AWS::EC2::Subnet::Id>
  Description: Two private subnets in different AZs — ASG instances sit here

CertificateArn:
  Type: String
  Description: IAM server certificate ARN for the HTTPS listener

RedisUrl:
  Type: String
  Description: ElastiCache Redis URL (redis://host:6379)

DynamoTable:
  Type: String
  Default: devops-nodes

DashboardToken:
  Type: String
  NoEcho: true
  Default: dev-token-change-me

AppPort:
  Type: Number
  Default: 3000

MinInstances:
  Type: Number
```

```
    Default: 1

  MaxInstances:
    Type: Number
    Default: 10

  DesiredInstances:
    Type: Number
    Default: 2

# ---------------------------------------------------------------------------
# Resources
# ---------------------------------------------------------------------------
Resources:

  # --- Security Groups -------------------------------------------------------

  ALBSecurityGroup:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: ALB - allow HTTP (redirect) and HTTPS from internet
      VpcId: !Ref VpcId
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 80
          ToPort: 80
          CidrIp: 0.0.0.0/0
        - IpProtocol: tcp
          FromPort: 443
          ToPort: 443
          CidrIp: 0.0.0.0/0
```

```
    Tags:
      - Key: Name
        Value: devops2-alb-sg

AppSecurityGroup:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: App instances - allow traffic from ALB and SSH
    VpcId: !Ref VpcId
    SecurityGroupIngress:
      # Only the ALB can reach the backend port
      - IpProtocol: tcp
        FromPort: !Ref AppPort
        ToPort: !Ref AppPort
        SourceSecurityGroupId: !Ref ALBSecurityGroup
      # SSH for debugging
      - IpProtocol: tcp
        FromPort: 22
        ToPort: 22
        CidrIp: 0.0.0.0/0
    Tags:
      - Key: Name
        Value: devops2-app-sg

# --- Load Balancer -------------------------------------------------------

LoadBalancer:
  Type: AWS::ElasticLoadBalancingV2::LoadBalancer
  Properties:
    Name: devops2-alb
    Scheme: internet-facing
```

```
    Type: application
    SecurityGroups:
      - !Ref ALBSecurityGroup
    Subnets: !Ref PublicSubnetIds
    Tags:
      - Key: Name
        Value: devops2-alb

  TargetGroup:
    Type: AWS::ElasticLoadBalancingV2::TargetGroup
    Properties:
      Name: devops2-tg
      Port: !Ref AppPort
      Protocol: HTTP
      TargetType: instance
      VpcId: !Ref VpcId
      HealthCheckPath: /health
      HealthCheckIntervalSeconds: 30
      HealthCheckTimeoutSeconds: 5
      HealthyThresholdCount: 2
      UnhealthyThresholdCount: 3
      # Stickiness so WebSocket clients reconnect to the same instance
      TargetGroupAttributes:
        - Key: stickiness.enabled
          Value: 'true'
        - Key: stickiness.type
          Value: lb_cookie
        - Key: stickiness.lb_cookie.duration_seconds
          Value: '86400'
      Tags:
        - Key: Name
```

```
      Value: devops2-tg

# HTTP on port 80 — redirects all traffic to HTTPS
HttpListener:
  Type: AWS::ElasticLoadBalancingV2::Listener
  Properties:
    LoadBalancerArn: !Ref LoadBalancer
    Port: 80
    Protocol: HTTP
    DefaultActions:
      - Type: redirect
        RedirectConfig:
          Protocol: HTTPS
          Port: '443'
          StatusCode: HTTP_301

# HTTPS on port 443 — terminates TLS, forwards to app instances on port 3000
HttpsListener:
  Type: AWS::ElasticLoadBalancingV2::Listener
  Properties:
    LoadBalancerArn: !Ref LoadBalancer
    Port: 443
    Protocol: HTTPS
    Certificates:
      - CertificateArn: !Ref CertificateArn
    DefaultActions:
      - Type: forward
        TargetGroupArn: !Ref TargetGroup

# --- Compute (Launch Template + ASG) -------------------------------------
```

```
LaunchTemplate:
  Type: AWS::EC2::LaunchTemplate
  Properties:
    LaunchTemplateName: devops2-lt
    LaunchTemplateData:
      ImageId: !Ref AmiId
      InstanceType: !Ref InstanceType
      KeyName: !Ref KeyName
      IamInstanceProfile:
        Name: LabInstanceProfile
      SecurityGroupIds:
        - !Ref AppSecurityGroup
      # Inject dynamic values that weren't in the AMI at bake time
      UserData:
        Fn::Base64: !Sub |
          #!/bin/bash
          ENV_FILE=/home/ec2-user/app/.env

          upsert_env() {
            local key=$1 val=$2
            if grep -q "^${!key}=" "$ENV_FILE" 2>/dev/null; then
              sed -i "s|^${!key}=.*|${!key}=${!val}|" "$ENV_FILE"
            else
              echo "${!key}=${!val}" >> "$ENV_FILE"
            fi
          }

          upsert_env REDIS_URL          "${RedisUrl}"
          upsert_env DYNAMO_TABLE_NODES "${DynamoTable}"
          upsert_env DASHBOARD_TOKEN    "${DashboardToken}"
```

```
        systemctl enable crond
        systemctl start crond
        systemctl enable devops2
        systemctl restart devops2

AutoScalingGroup:
  Type: AWS::AutoScaling::AutoScalingGroup
  DependsOn: HttpsListener
  Properties:
    AutoScalingGroupName: devops2-asg
    MinSize: !Sub '${MinInstances}'
    MaxSize: !Sub '${MaxInstances}'
    DesiredCapacity: !Sub '${DesiredInstances}'
    LaunchTemplate:
      LaunchTemplateId: !Ref LaunchTemplate
      Version: !GetAtt LaunchTemplate.LatestVersionNumber
    TargetGroupARNs:
      - !Ref TargetGroup
    VPCZoneIdentifier: !Ref PrivateSubnetIds
    HealthCheckType: ELB
    HealthCheckGracePeriod: 120
    Tags:
      - Key: Name
        Value: devops2-node
        PropagateAtLaunch: true

# --- Step Scaling Policies ----------------------------------------------

ScaleOutPolicy:
  Type: AWS::AutoScaling::ScalingPolicy
  Properties:
```

```
    AutoScalingGroupName: !Ref AutoScalingGroup
    PolicyName: devops2-scale-out
    PolicyType: StepScaling
    AdjustmentType: ChangeInCapacity
    MetricAggregationType: Average
    # Step 1: moderate load  → add 1 instance
    # Step 2: heavy load     → add 2 instances
    StepAdjustments:
      - MetricIntervalLowerBound: 0
        MetricIntervalUpperBound: 50
        ScalingAdjustment: 1
      - MetricIntervalLowerBound: 50
        ScalingAdjustment: 2

  ScaleInPolicy:
    Type: AWS::AutoScaling::ScalingPolicy
    Properties:
    AutoScalingGroupName: !Ref AutoScalingGroup
    PolicyName: devops2-scale-in
    PolicyType: StepScaling
    AdjustmentType: ChangeInCapacity
    MetricAggregationType: Average
    StepAdjustments:
      - MetricIntervalUpperBound: 0
        ScalingAdjustment: -1

  # --- CloudWatch Alarms ----------------------------------------------------

  HighRequestAlarm:
    Type: AWS::CloudWatch::Alarm
    Properties:
```

```
    AlarmName: devops2-high-requests
    AlarmDescription: Request count per target high — scale out
    Namespace: AWS/ApplicationELB
    MetricName: RequestCountPerTarget
    Statistic: Sum
    Period: 60
    EvaluationPeriods: 1
    # Fire when requests/target/minute >= 100
    Threshold: 100
    ComparisonOperator: GreaterThanOrEqualToThreshold
    Dimensions:
      - Name: TargetGroup
        Value: !GetAtt TargetGroup.TargetGroupFullName
    AlarmActions:
      - !Ref ScaleOutPolicy
    TreatMissingData: notBreaching

  LowRequestAlarm:
    Type: AWS::CloudWatch::Alarm
    Properties:
    AlarmName: devops2-low-requests
    AlarmDescription: Request count per target low — scale in
    Namespace: AWS/ApplicationELB
    MetricName: RequestCountPerTarget
    Statistic: Sum
    Period: 300
    EvaluationPeriods: 2
    # Fire when requests/target/5min < 20 for two consecutive periods
    Threshold: 20
    ComparisonOperator: LessThanThreshold
    Dimensions:
```

```
      - Name: TargetGroup
        Value: !GetAtt TargetGroup.TargetGroupFullName
    AlarmActions:
      - !Ref ScaleInPolicy
    TreatMissingData: notBreaching

# ---------------------------------------------------------------------------
# Outputs
# ---------------------------------------------------------------------------
Outputs:

  LoadBalancerDNS:
    Description: ALB public DNS — use this as VITE_API_URL when rebuilding frontend
    Value: !GetAtt LoadBalancer.DNSName
    Export:
      Name: devops2-alb-dns

  TargetGroupArn:
    Value: !Ref TargetGroup
    Export:
      Name: devops2-tg-arn

  AutoScalingGroupName:
    Value: !Ref AutoScalingGroup
    Export:
      Name: devops2-asg-name

  AppSecurityGroupId:
    Description: SG attached to ASG instances — must be authorised in the Redis SG
    Value: !Ref AppSecurityGroup
    Export:
```

```
    Name: devops2-app-sg-id

ScaleOutPolicyArn:
  Value: !Ref ScaleOutPolicy

ScaleInPolicyArn:
  Value: !Ref ScaleInPolicy
```